

Arbore Indexat Binar

Variante și implementări

Andrei Mihai Ciobanu

Algolymp Summer School

AIB Clasic — Point add, range sum

Range add, point query

Range add, range sum

Prefix minimum

AIB 2D

Recapitulare

AIB Clasic — Point add, range sum

Problema de bază

Problemă

Dat un şir $a_1, \dots, a_n$. Se efectuează q operații:

- **Point add:** adaugă x la a_i
- **Range sum:** calculează $\sum_{j=l}^r a_j$

Abordări naive:

Metodă	Update	Query
Array simplu	$O(1)$	$O(n)$
Sume parțiale	$O(n)$ (recompilare)	$O(1)$
AIB	$O(\log n)$	$O(\log n)$

Soluția cu sume parțiale e $O(1)$ la query dar la fiecare update trebuie refăcute $O(n)$ valori — pentru $q = 10^5$ operații mixte, TLE.

Bitul cel mai puțin semnificativ

lsb(i) — Least Significant Bit

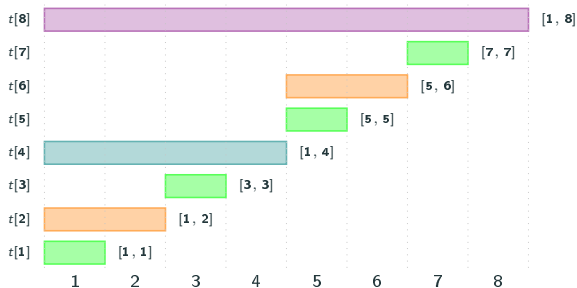
Cel mai mic bit setat al lui i :

$$\text{lsb}(i) = i \ \& \ (-i)$$

În complement față de 2, $-i$ inversează toți biții și adaugă 1. AND-ul cu i izolează **singurul bit rămas în poziție identică**.

i	binar	$-i$ (compl. 2)	lsb(i)
6	0110	...1010	2
8	1000	...1000	8
5	0101	...1011	1
12	1100	...0100	4

$t[i]$ stochează suma elementelor din intervalul $[i - \text{lsb}(i) + 1, i]$.



■ lsb = 1 ■ lsb = 2 ■ lsb = 4 ■ lsb = 8 Lungimea barei = numărul de elemente acoperite.

Query — Prefix sum $[1, i]$

Idee: adunăm $t[i]$, sărim la $i - \text{lsb}(i)$, repetăm până la 0.

Traseu pentru $\text{get}(7)$

- $i = 7$: $t[7] = a[7]$; $i \leftarrow 7 - 1 = 6$
- $i = 6$: $t[6] = a[5] + a[6]$; $i \leftarrow 6 - 2 = 4$
- $i = 4$: $t[4] = a[1] + \dots + a[4]$; $i \leftarrow 4 - 4 = 0$ — stop

Suma returnată: $a[1] + \dots + a[7]$ ✓

Complexitate: $O(\log n)$ — la fiecare pas se elimină un bit setat.

Suma pe interval $[l, r]$: $\text{get}(r) - \text{get}(l-1)$.

Update — Point add la poziția i

Idee: actualizăm $t[i]$, sărim la $i + \text{lsb}(i)$, repetăm până depășim n .

Traseu pentru $\text{add}(3, x)$

- $i = 3$: $t[3] += x$; $i \leftarrow 3 + 1 = 4$
- $i = 4$: $t[4] += x$; $i \leftarrow 4 + 4 = 8$
- $i = 8$: $t[8] += x$; $i \leftarrow 8 + 8 = 16 > n$ — stop

Actualizăm exact pozițiile al căror interval conține indexul 3: $t[3]$ (acoperă $[3, 3]$), $t[4]$ (acoperă $[1, 4]$), $t[8]$ (acoperă $[1, 8]$).

Complexitate: $O(\log n)$ — la fiecare pas se adaugă un bit.

Implementare AIB Clasic

```
1  const int nmax = 100005;
2  long long t[nmax];
3  int n;
4
5  void add(int i, long long x) {
6      for (; i <= n; i += i & -i) {
7          t[i] += x;
8      }
9  }
10
11 long long get(int i) {
12     long long s = 0;
13     for (; i > 0; i -= i & -i) {
14         s += t[i];
15     }
16     return s;
17 }
18
19 long long get(int l, int r) {
20     return get(r) - get(l - 1);
21 }
```

Range add, point query

Range add, point query — Idee

Problemă

- **Range add:** adaugă x la toate a_i cu $l \leq i \leq r$
- **Point query:** care e valoarea curentă a lui a_i ?

Transformare cu șir de diferențe: fie $d[i] = a[i] - a[i - 1]$. Atunci:

$$a[i] = \sum_{j=1}^i d[j] \quad \Rightarrow \quad \text{point query} = \text{prefix sum pe } d$$

Operații pe d :

- Range add $[l, r]$ cu x : $d[l] += x$, $d[r+1] -= x$ — 2 actualizări punct
- Point query $a[i]$: prefix sum $d[1..i]$ cu $\text{get}(i)$

Menținem un **AIB pe șirul de diferențe d** . Ambele operații: $O(\log n)$.

Range add, point query — Implementare

```
1  const int nmax = 100005;
2  long long d[nmax]; // AIB pe sirul de diferente
3  int n;
4
5  void range_add(int l, int r, long long x) {
6      for (int i = l; i <= n; i += i & -i) {
7          d[i] += x;
8      }
9      for (int i = r + 1; i <= n; i += i & -i) {
10         d[i] -= x;
11     }
12 }
13
14 long long point_get(int i) {
15     long long s = 0;
16     for (; i > 0; i -= i & -i) {
17         s += d[i];
18     }
19     return s;
20 }
```

Range add, range sum

Range add, range sum — De ce un singur AIB nu ajunge

Știm deja: range add + **point query** → AIB pe șirul de diferențe d .

$a[i] = d.get(i)$. Dar **range sum** $[l, r]$?

$$P(i) = \sum_{j=1}^i a[j] = \sum_{j=1}^i \sum_{k=1}^j d[k] = \sum_{k=1}^i d[k] \cdot (i - k + 1)$$

Rezultatul depinde atât de i cât și de *care* $d[k]$ sunt nenuli — nu putem calcula $P(i)$ dintr-o singură interogare pe AIB.

Soluție

Scriem $P(i)$ ca diferența a două prefix sums, fiecare menținut într-un AIB:

$$P(i) = \underbrace{B_1.get(i)}_{\text{coef. lui } i} \cdot i - \underbrace{B_2.get(i)}_{\text{corecție constantă}}$$

Actualizăm B_1 și B_2 în $O(\log n)$ la fiecare range add.

Range add, range sum — Derivarea lui B_1 și B_2

Ce se întâmplă cu $P(i)$ după $\text{range_add}(l, r, x)$?

$i < l$	$l \leq i \leq r$	$i > r$
$\Delta P = 0$	$\Delta P = x(i-l+1) = x \cdot i - x(l-1)$	$\Delta P = x(r-l+1)$ (constant)

Dacă actualizăm B_1, B_2 astfel:

$$B_1.\text{add}(l, x), \quad B_1.\text{add}(r+1, -x) \quad B_2.\text{add}(l, x(l-1)), \quad B_2.\text{add}(r+1, -x \cdot r)$$

atunci $P(i) = B_1.\text{get}(i) \cdot i - B_2.\text{get}(i)$:

	$B_1.\text{get}(i)$	$B_2.\text{get}(i)$	$B_1 \cdot i - B_2$
$i < l$	0	0	0 ✓
$l \leq i \leq r$	x	$x(l-1)$	$xi - x(l-1) = x(i-l+1)$ ✓
$i > r$	0	$x(l-1-r)$	$-x(l-1-r) = x(r-l+1)$ ✓

Range sum: $\text{sum}(l, r) = P(r) - P(l-1)$.

Range add, range sum — Implementare

```
1 // add_aib / get_aib: aceleasi ca add/get clasic, cu AIB ca parametru
2 long long b1[nmax], b2[nmax];
3 int n;
4
5 void range_add(int l, int r, long long x) {
6     add_aib(b1, l, x);    add_aib(b1, r + 1, -x);
7     add_aib(b2, l, x*(l-1)); add_aib(b2, r + 1, -x*r);
8 }
9
10 long long prefix_sum(int i) {
11     return get_aib(b1, i) * i - get_aib(b2, i);
12 }
13
14 long long range_sum(int l, int r) {
15     return prefix_sum(r) - prefix_sum(l - 1);
16 }
```

Prefix minimum

Prefix minimum — Idee și limitări

Problemă

Dat un șir a . Operații:

- **Update:** setează $a[i] = x$ (valoare mai mică decât cea curentă)
- **Prefix min:** calculează $\min(a[1], \dots, a[i])$

Implementare aproape identică cu AIB-ul clasic, dar cu min în loc de +:

- $t[i]$ stochează min pe intervalul $[i - \text{lsb}(i) + 1, i]$
- Query: parcurgem ca la sum, dar luăm minimumul
- Update: actualizăm $t[i] = \min(t[i], x)$ și sărim înainte

Limitări față de AIB cu sumă

- **Doar prefix** $[1, i]$, **nu range** $[l, r]$ — min nu e invertibil
- Update-urile trebuie să fie **monoton descrescătoare** ($x \leq a[i]$ curent)

Prefix minimum — Implementare

```
1  const int nmax = 100005;
2  const long long INF = 1e18;
3  long long t[nmax];
4  int n;
5
6  // Initializare: t[i] = INF pentru toti i
7
8  void update(int i, long long x) {
9      for (; i <= n; i += i & -i) {
10         t[i] = min(t[i], x);
11     }
12 }
13
14 long long prefix_min(int i) {
15     long long res = INF;
16     for (; i > 0; i -= i & -i) {
17         res = min(res, t[i]);
18     }
19     return res;
20 }
```

AIB 2D

Problemă

Matrice $n \times m$. Operații:

- **Point add:** adaugă x la $a[i][j]$
- **Prefix 2D sum:** $\sum_{r \leq i, c \leq j} a[r][c]$

Idee: aplicăm același salt cu lsb pe **ambele dimensiuni**.

$t[i][j]$ stochează suma pe submatricea $[i - \text{lsb}(i) + 1, i] \times [j - \text{lsb}(j) + 1, j]$.

Complexitate: $O(\log n \cdot \log m)$ per operație. **Suma pe submatrice**

$[r_1, c_1] \times [r_2, c_2]$ prin incluziune-excludere:

$$\text{get}(r_2, c_2) - \text{get}(r_2, c_1 - 1) - \text{get}(r_1 - 1, c_2) + \text{get}(r_1 - 1, c_1 - 1)$$

```
1  const int nmax = 1005;
2  long long t[nmax][nmax];
3  int n, m;
4
5  void add(int i, int j, long long x) {
6      for (; i <= n; i += i & -i) {
7          for (int y = j; y <= m; y += y & -y) { t[i][y] += x; }
8      }
9  }
10
11 long long get(int i, int j) {
12     long long s = 0;
13     for (; i > 0; i -= i & -i) {
14         for (int y = j; y > 0; y -= y & -y) { s += t[i][y]; }
15     }
16     return s;
17 }
18
19 long long get(int r1, int c1, int r2, int c2) {
20     return get(r2,c2) - get(r2,c1-1)
21         - get(r1-1,c2) + get(r1-1,c1-1);
22 }
```

Recapitulare

VARIANTĂ	Update	Query	Condiție
Point add, range sum	$O(\log n)$	$O(\log n)$	—
Range add, point query	$O(\log n)$	$O(\log n)$	—
Range add, range sum	$O(\log n)$	$O(\log n)$	—
Prefix minimum	$O(\log n)$	$O(\log n)$	update monoton $\downarrow$
AIB 2D	$O(\log^2 n)$	$O(\log^2 n)$	—

AIB vs. Segment Tree:

- + Implementare mai scurtă, constantă mai mică, memorie exactă $O(n)$
- Funcționează **doar cu operații inversabile** (sumă, XOR)
- Range min/max, lazy propagation $\rightarrow$ Segment Tree

Probleme de antrenament

1. **Point Add Range Sum** (CSES #1648 — AIB clasic)
2. **Range Update Queries** (CSES #1651 — range add, point query)
3. **Forest Queries II** (CSES #1739 — AIB 2D)
4. **Increasing Subsequence** (CSES #1145 — SCLM cu AIB prefix min)
5. **AIB** (infoarena — problemă clasică)
6. **perm** (Kilonova #2669 — ONI 2024 Baraj Seniori)

1. **CP-Algorithms** — Fenwick Tree (EN, foarte complet cu dovezi)
2. **USACO Guide** — Point Update Range Sum (Gold)
3. **Brilliant** — Fenwick Tree (EN, accesibil)
4. **Algorithmica** — AIB vs. Segment Tree cu benchmark-uri